



**Relatório Final do Sistema de Navegação Autônoma
Hierárquica em CoppeliaSim: Integração de Percepção por
Visão, Planejamento Global A* e Controle Local DWA**

Discente: Adhemar Luiz Cavalcanti Silva Rocha Neto
Docente: Juracy Emanuel Magalhães

Repositório Oficial: github.com/AdhemarCavalcanti/robotica_projeto_final

1. Resumo Executivo

Este documento apresenta o relatório de desenvolvimento de um sistema de navegação autônoma e desvio de obstáculos para um robô móvel em ambiente simulado. A solução adota uma abordagem hierárquica dividida em três camadas principais: **percepção ambiental** (por mapeamento de ocupação visual e sensores de proximidade), **planejamento global** (algoritmo A* modificado com suavização Bezier e simplificação de linha de visada) e **controle local** (Algoritmo de Janela Dinâmica - DWA). O sistema demonstrou robustez em cenários com múltiplos obstáculos e capacidade de recuperação contra travamentos (*stuck*), garantindo a chegada segura ao alvo.

2. Arquitetura Estrutural do Software

O projeto foi reestruturado de forma modular para garantir manutenibilidade e separação de conceitos fundamentais em robótica móvel:

- [mapa_ocupacao.py](#) (**Módulo de Percepção Estática**): Instancia um sensor de visão ortogonal no CoppeliaSim, lê os dados de profundidade (ou intensidade como *fallback*) e gera uma grade de ocupação discreta convertida para coordenadas globais do mundo.
- [dwa_navigation.py](#) (**Módulo de Inteligência e Navegação**): Contém as classes [AStarPlanner](#) (planejador global que processa a malha gerada pela visão, simplifica curvas colineares e suaviza a trajetória) e [DWAController](#) (gerador de comandos de velocidade angular e linear considerando limites físicos e proximidade de obstáculos).
- [main_robo.py](#) (**Módulo de Orquestração**): Loop principal de simulação sincronizada. Gerencia os estados do robô, realiza chamadas de replanejamento se o veículo ficar preso e traduz velocidades (linear e angular) para atuação nas rodas (*unicycle-to-differential model*).

3. Cronograma de Evolução e Histórico de Commits

O ciclo de desenvolvimento do repositório reflete uma metodologia ágil e incremental, dividida em quatro fases macro:

Fase 1: Fundação, Estruturação e Comportamento Primitivo (Maio / 2026)

- **05 de Maio:** *Primeiro commit, atualizações de pacotes e a crucial reestruturação de projeto para separar o módulo de comportamento.* O código nasce de forma limpa, isolando controle de cinemática de outras funções.

- **05 e 06 de Maio:** *Implementação da lógica de percepção e revisão de código de janela dinâmica.* Inicialização do modelo matemático do DWA focado em predição de trajetória.
- **22 de Maio:** *Implementação com DWA funcional, indo até o alvo e implementação de stop no alvo.* O controle cinemático local prova-se estável, permitindo ao robô desacelerar e estacionar ao atingir o limiar de proximidade de 0.20 m.
- **28 de Maio:** *Melhoramento de tratamentos de gargalos.* Adição de heurísticas de mitigação para evitar oscilações locais típicas do DWA em quinas.

Fase 2: Configuração de Cenário e Integração Física (Finais de Maio / 2026)

- **29 de Maio:** *Cena adicionada ao projeto.* O robô é colocado em um mapa oficial do CoppeliaSim com paredes e formas geométricas complexas, exigindo uma abordagem de mapeamento global.

Fase 3: Maturidade de Controle e Mapeamento de Ocupação (Junho / 2026)

- **18 de Junho:** *Melhoramento no dwa e dwa mais sensível.* Ajuste fino nas funções de custo de obstáculos (α , β e γ), aproximando a margem de colisão para os limites geométricos físicos do robô ($r = 0.24$ m, margem 0.06 m).
- **30 de Junho:** *Mapa implementado de obstáculos.* Transição bem-sucedida para o uso da biblioteca [numpy](#) para extrair os pontos ocupados da cena por projeção de profundidade do sensor superior.

Fase 4: Ajustes Geométricos Finais e Homologação (Julho / 2026)

- **02 de Julho:** *Cena atualizada com sensor e eixos corrigidos.* Correção matemática na transposição e rotação da matriz do sensor ([flip_y=True](#)), alinhando o grid bidimensional retornado com as coordenadas físicas de mundo X/Y do simulador.
- **05 de Julho:** *Projeto final finalizado.* Consolidação das rotinas de *recovery* (atuação em marcha ré para desvencilhar-se de cantos mortos), paralelismo (*multi-threading*) para exibição de mapas com [matplotlib](#) sem travar o loop de simulação física, e validação geral do sistema.

4. Análise Algorítmica da Solução Implementada

A. Planejamento Global (A* Modificado)

O [AStarPlanner](#) mapeia a cena utilizando um algoritmo de busca em grafo discreto de 8 direções. Possui recursos avançados de engenharia de robótica:

1. **Inflação de Obstáculos:** As células da grade num raio correspondente ao tamanho do robô ([inf_cells](#)) são marcadas como bloqueadas, garantindo que o centro do robô nunca planeje caminhos raspando nas quinas.
2. **Otimização LOS (Line-of-Sight) e Bézier:** Após achar o caminho por nós discretos, o planejador aplica a técnica *Line-of-Sight* para remover nós colineares desnecessários. Em seguida, aplica curvas de Bézier cúbicas nos pontos de inflexão para suavizar curvas fechadas de 90°, tornando o caminho condizente com as capacidades de curvas do robô real.

B. Controle Local (Dynamic Window Approach - DWA)

O método avalia um espaço de busca centrado nas velocidades admissíveis e seguras em uma janela temporal reduzida ($\Delta t = 0.1$ s):

A escolha dos comandos ótimos de velocidade (v, ω) é realizada minimizando uma função de custo multifatorial:

- **cost_goal**: Alinha a orientação frontal do robô com o objetivo.
- **cost_obs**: Aplica uma penalidade inversamente proporcional à distância do obstáculo mais próximo. Se cruzar o raio de colisão, o custo torna-se infinito (∞).
- **cost_dist**: Minimiza a distância euclidiana restante até o ponto alvo local.

C. Mecanismo de Recuperação contra Travamentos (*Stuck*)

Em situações onde o DWA entra em mínimos locais (como um robô encurralado de frente para uma parede em formato de "U"), o script `main_robo.py` executa um contador de passos estáticos (`_stk_steps`). Se o robô não mudar substancialmente de posição por 30 iterações de loop, o sistema ativa uma sub-rotina de emergência (`recovery_cmd`), ordenando uma manobra combinada de marcha ré seguida por rotação sobre o próprio eixo para livrar a frente do robô do obstáculo, forçando em seguida o replanejamento global do A^* .

5. Conclusão

O projeto foi concluído com absoluto sucesso. O código reflete práticas maduras de programação em Python aplicadas à engenharia de controle, tais como vetorização de matrizes com `numpy`, tratamento de exceções robusto para chamadas remotas de API e uso seguro de `threads` para interface gráfica. O encadeamento histórico dos *commits* demonstra um processo estruturado e consciente de desenvolvimento que resultou em um robô móvel simulado autônomo e altamente resiliente.